ELK Stack Lab - Chapter 5

Student Name	Kara Mohamed Mourtadha	**Group**	5
Course	Systems Reliability Engineering (SRE)	**Class**	Semester 1, 2025-2026
Lab	ELK Stack - Observability & Centralized Logging	**Chapter**	5

Section 1 — The Challenge

1.1 Why did binary "up/down" monitoring fail?

Binary monitoring failed because it only answers "Is the service reachable?" not "How well is it performing?"

Three reasons predefined questions fail for p99 latency:

- **Cardinality explosion:** Latency depends on service, endpoint, payload, zone, dependencies. No health-check predicts all combinations.
- **Unknown unknowns:** p99 spikes come from cross-service interactions invisible to individual service checks.
- **Statistical invisibility:** Mean hides outliers. Only p95/p99 and traces catch slow requests.

Section 2 — The Three Pillars of Observability

2.1 Pillar Characteristics Table

Pillar	Data Characteristic	E-Commerce Use Case
Logs	Discrete events, high cardinality. Immutable timestamped records.	Debug failed checkout by order_id
Metrics	Aggregated numeric time-series, low cardinality. Sampled at fixed intervals.	Alert error_rate > 2% over 5min
Traces	Per-request call graphs, very high cardinality. DAG of spans with trace_id.	Find which service caused 3s delay

2.2 How do Traces resolve bottlenecks?

Traces resolve bottlenecks through **end-to-end correlation via trace ID**:

- **Propagation:** trace_id injected into every downstream call.
- **Span recording:** Each service records timed spans with parent-child relationships.
- **Waterfall:** Spans assembled showing exactly where delays occur across services.

Section 3 — Part 2: ELK Technical Analysis

3.1 Beats & Elastic Agent

Beats are lightweight Go shippers running as OS agents.

- **Filebeat** — tails log files, tracks positions in registry.
- **Winlogbeat** — subscribes to Windows Event Log API.

Elastic Agent — unified shipper managed via Kibana Fleet.

3.2 Logstash & Grok

Logstash — Input (port 5044) → Filter (Grok/GeoIP) → Output (ES).

Grok — parses text into structured fields using named regex patterns.

3.3 Elasticsearch & The Inverted Index

Elasticsearch — distributed search engine using inverted index.

Inverted index maps "word → document IDs" enabling sub-millisecond search across terabytes.

3.4 Kibana

Kibana — visualization layer: Discover, Visualize, Dashboard, Maps.

Section 4 — Part 3: WEF Design Challenge

4.1 WEF Architecture Evaluation Table

Feature	Source-Initiated (Push)	Collector-Initiated (Pull)
Scalability	HIGH — agents push independently	LOW — collector polls each source
Firewall	Complex — inbound rules needed	Simple — outbound only
Remote Laptops	IDEAL — tolerates disconnection	PROBLEMATIC — cannot poll offline

4.2 Windows Server 2025 Innovation

> **IAKerb** — proxies Kerberos auth through intermediary, eliminating NTLM fallback.
>
> **Local KDC** — embedded KDC for offline Kerberos auth using cached credentials.

4.3 Forensic Prioritization

Identify the two specific Windows Event IDs for logon success and failure:

> **Logon Success Event ID: 4624**
>
> **Logon Failure Event ID: 4625**

Section 5 — Part 4: SLIs and Error Budgets

5.1 Error Budget Calculation

Given: 99.95% SLO · Total minutes in 30 days = 43,200

Step 1 — SLO fraction	`SLO = 99.95% = 0.9995`
Step 2 — Unavailability	`1 - SLO = 0.0005`
Step 3 — Error Budget (min)	`43,200 x 0.0005 = 21.6 minutes/month`

Error Budget = 21.6 minutes per month

5.2 SLI Proposal

Proposed SLI: Percentage of POST /api/checkout requests with HTTP 2xx AND latency < 500ms.

- **HTTP 2xx gate:** 5xx = complete checkout failure.
- **500ms latency gate:** Cart abandonment rises above 400ms.

5.3 The "SRE Clamp"

When >50% of error budget consumed (>10.8 of 21.6 min): **freeze feature deployments**, redirect to reliability work.

Section 6 — Part 5: Practical Implementation

6.1 docker-compose.yml

```
version: '3.8'

networks:
  elk-network:
    driver: bridge
    ipam:
      config:
        - subnet: 172.20.0.0/16

volumes:
  es-data:
    driver: local

services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.12.0
    container_name: elk-elasticsearch
    environment:
      - node.name=es-node-1
      - cluster.name=elk-cluster
      - discovery.type=single-node
      - bootstrap.memory_lock=true
      - ES_JAVA_OPTS=-Xms1g -Xmx1g
      - xpack.security.enabled=false
    ulimits:
      memlock:
        soft: -1
        hard: -1
    volumes:
      - es-data:/usr/share/elasticsearch/data
    ports:
      - "9200:9200"
    networks:
```

```
      - elk-network
    healthcheck:
      test: ["CMD-SHELL", "curl -s http://localhost:9200 | grep -q cluster_name"]
      interval: 30s
      timeout: 10s
      retries: 5
    restart: unless-stopped
```

```
  logstash:
    image: docker.elastic.co/logstash/logstash:8.12.0
    container_name: elk-logstash
    volumes:
      - ../config/logstash.conf:/usr/share/logstash/pipeline/logstash.conf:ro
    ports:
      - "5044:5044"
    depends_on:
      elasticsearch:
        condition: service_healthy
    networks:
      - elk-network
    restart: unless-stopped

  kibana:
    image: docker.elastic.co/kibana/kibana:8.12.0
    container_name: elk-kibana
    ports:
      - "5601:5601"
    depends_on:
      elasticsearch:
        condition: service_healthy
    networks:
      - elk-network
    healthcheck:
      test: ["CMD-SHELL", "curl -s http://localhost:5601/api/status | grep -q available"]
      interval: 30s
      timeout: 10s
      retries: 5
```

```
    restart: unless-stopped

  nginx-lb:
    image: nginx:1.25-alpine
    container_name: nginx-lb
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ../html:/usr/share/nginx/html:ro
      - ../logs/nginx:/var/log/nginx
    ports:
      - "80:80"
    networks:
      - elk-network
    restart: unless-stopped

  app-service:
    build:
      context: ./app
      dockerfile: ./Dockerfile
    container_name: app-service
    volumes:
      - ../logs/app:/var/log/app
    ports:
      - "5000:5000"
    networks:
      - elk-network
    restart: unless-stopped

  ssh-simulator:
    image: ubuntu:22.04
    container_name: ssh-simulator
    volumes:
      - ./generate_ssh_logs.sh:/generate_ssh_logs.sh:ro
      - ../logs/ssh:/var/log/ssh
    entrypoint: ["/bin/bash", "-c", "apt-get update -qq && apt-get install -y -qq python3 curl && chmod +x /g
    networks:
      - elk-network
    restart: unless-stopped
```

```
  syslog-simulator:
    image: ubuntu:22.04
    container_name: syslog-simulator
    volumes:
      - ./generate_syslog.sh:/generate_syslog.sh:ro
      - ../logs/syslog:/var/log/syslog
    entrypoint: ["/bin/bash", "-c", "apt-get update -qq && apt-get install -y -qq python3 curl && chmod +x /g
    networks:
      - elk-network
    restart: unless-stopped
```

```
  filebeat:
    image: docker.elastic.co/beats/filebeat:8.12.0
    container_name: filebeat
    user: root
    volumes:
      - ../config/filebeat.yml:/usr/share/filebeat/filebeat.yml:ro
      - ../logs:/var/log:ro
      - /var/run/docker.sock:/var/run/docker.sock:ro
    depends_on:
      - logstash
    networks:
      - elk-network
    restart: unless-stopped
```

6.2 filebeat.yml

```
filebeat.inputs:
  - type: filestream
    id: nginx-access
    enabled: true
    paths:
      - /var/log/nginx/access.log
    fields:
      service: nginx
    fields_under_root: true
```

```
    harvester_buffer_size: 16384
    max_bytes: 1048576
    close_inactive: 1m
    scan_frequency: 1s

  - type: filestream
    id: nginx-error
    enabled: true
    paths:
      - /var/log/nginx/error.log
    fields:
      service: nginx
    fields_under_root: true
    harvester_buffer_size: 16384
    max_bytes: 1048576

  - type: filestream
    id: ssh-auth
    enabled: true
    paths:
      - /var/log/ssh/auth.log
    fields:
      service: ssh
    fields_under_root: true
    harvester_buffer_size: 16384
    max_bytes: 1048576
    parsers:
      - multiline:
          type: pattern
          pattern: '^\{'
          negate: true
```

```
          match: after

  - type: filestream
    id: syslog-linux
    enabled: true
    paths:
      - /var/log/syslog/linux.log
```

```
    fields:
      service: syslog
    fields_under_root: true
    harvester_buffer_size: 16384
    max_bytes: 1048576

  - type: filestream
    id: app-requests
    enabled: true
    paths:
      - /var/log/app/requests.log
    fields:
      service: app
    fields_under_root: true
    harvester_buffer_size: 16384
    max_bytes: 1048576
    parsers:
      - multiline:
          type: pattern
          pattern: '^\{'
          negate: true
          match: after
          max_lines: 100
          timeout: 5s

filebeat.config.modules:
  path: ${path.config}/modules.d/*.yml
  reload.enabled: false

processors:
  - add_host_metadata:
      when.not.contains.tags: forwarded
  - add_cloud_metadata: ~
```

```
  - add_docker_metadata: ~

output.logstash:
  hosts: ["logstash:5044"]
  bulk_max_size: 2048
```

```
  compression_level: 3
  worker: 2

logging.level: info
logging.to_files: true
logging.files:
  path: /var/log/filebeat
  name: filebeat
  keepfiles: 7
  rotateeverybytes: 10485760
```

6.3 winlogbeat.yml

```
winlogbeat.event_logs:
  - name: Security
    processors:
      - script:
          when.equals.event_id: 4624
          fields:
            logon_result: success
      - script:
          when.equals.event_id: 4625
          fields:
            logon_result: failure
  - name: System
  - name: Application

fields:
  env: lab
  datacenter: dc1
fields_under_root: true

output.logstash:
  hosts: ["elk-server:5044"]
  compression_level: 3

logging.level: info
logging.to_files: true
logging.files:
```

```
  path: /var/log/winlogbeat
  name: winlogbeat
  keepfiles: 7
```

6.4 logstash.conf

```
input {
  beats {
    port => 5044
  }
}

filter {
  if [service] == "nginx" {
    grok {
      match => { "message" => "%{COMBINEDAPACHELOG}" }
      overwrite => ["message"]
      add_field => { "[@metadata][index]" => "nginx" }
    }
    date {
      match => [ "timestamp", "dd/MMM/yyyy:HH:mm:ss Z" ]
      target => "@timestamp"
    }
    mutate {
      convert => { "status" => "integer" "bytes" => "integer" }
    }
    geoip {
      source => "clientip"
      target => "geoip"
      add_field => { "country" => "%{[geoip][country_name]}" }
    }
  }

  if [service] == "ssh" {
    json {
      source => "message"
      target => "ssh_data"
```

```
    }
    date {
      match => [ "[ssh_data][timestamp]", "ISO8601" ]
      target => "@timestamp"
    }
    mutate {
      rename => { "[ssh_data][user]" => "ssh_user" }
      rename => { "[ssh_data][source_ip]" => "ssh_ip" }
      rename => { "[ssh_data][result]" => "ssh_result" }
      rename => { "[ssh_data][country]" => "ssh_country" }
      add_field => { "[@metadata][index]" => "ssh" }
      remove_field => ["ssh_data"]
    }
    geoip {
      source => "ssh_ip"
      target => "geoip"
      add_field => { "ssh_country" => "%{[geoip][country_name]}" }
    }
  }

  if [service] == "syslog" {
    grok {
      match => { "message" => "<%{POSINT:priority}>%{NONZERO_INT:version} %{TIMESTAMP_ISO8601:timestamp} %{HO
      overwrite => ["message"]
      add_field => { "[@metadata][index]" => "syslog" }
    }
    mutate {
      convert => { "priority" => "integer" }
    }
  }

  if [service] == "app" {
    json {
      source => "message"
      target => "app_data"
    }
    date {
```

```
      match => [ "[app_data][timestamp]", "ISO8601" ]
      target => "@timestamp"
    }
    mutate {
      rename => { "[app_data][method]" => "request_method" }
      rename => { "[app_data][endpoint]" => "request_endpoint" }
      rename => { "[app_data][status_code]" => "request_status" }
      rename => { "[app_data][latency_ms]" => "request_latency" }
      add_field => { "[@metadata][index]" => "app" }
      remove_field => ["app_data"]
    }
  }
```

```
  mutate {
    add_field => { "lab" => "elk-observability" }
  }
}

output {
  elasticsearch {
    hosts => ["elasticsearch:9200"]
    index => "logs-%{[@metadata][index]}-%{+YYYY.MM.dd}"
    manage_template => false
  }
  stdout {
    codec => rubydebug
  }
}
```

Section 7 — Final Deliverable: System Interaction Maps

Data Source 1 — Nginx Web Server

Source Name	Nginx Web Server
Telemetry Type	Log
Shipper	Filebeat
Identity Protocol	Combined Apache Log format via Beats on TCP 5044
Processing	Logstash Grok filter %{COMBINEDAPACHELOG}
Transformation	Extract clientip, status, request. GeoIP enrichment.
Final Destination	Elasticsearch: logs-nginx-YYYY.MM.DD
Visualization	Kibana — Traffic charts, status codes, geo maps

Data Source 2 — SSH Authentication Events

Source Name	SSH Authentication Simulator
Telemetry Type	Log
Shipper	Filebeat
Identity Protocol	JSON via Beats on TCP 5044
Processing	Logstash JSON filter
Transformation	Extract user, result, source_ip. GeoIP enrichment.
Final Destination	Elasticsearch: logs-ssh-YYYY.MM.DD
Visualization	Kibana — Auth results pie chart, attack map

Data Source 3 — Syslog Linux (RFC5424)

Source Name	Linux Syslog Simulator
Telemetry Type	Log
Shipper	Filebeat
Identity Protocol	RFC5424 via Beats on TCP 5044
Processing	Logstash Grok filter
Transformation	Extract priority, hostname, process, message.
Final Destination	Elasticsearch: logs-syslog-YYYY.MM.DD
Visualization	Kibana — System events timeline

Data Source 4 — App Service (REST API)

Source Name	Flask App Service
Telemetry Type	Log + Metric
Shipper	Filebeat
Identity Protocol	JSON via Beats on TCP 5044
Processing	Logstash JSON filter
Transformation	Extract method, endpoint, status_code, latency_ms.
Final Destination	Elasticsearch: logs-app-YYYY.MM.DD
Visualization	Kibana — Latency histogram, endpoint counts

Data Source 5 — Windows Security Event Logs

Source Name	Windows Server 2025 Security Event Log
Telemetry Type	Log
Shipper	Winlogbeat
Identity Protocol	Windows Event XML via Beats on TCP 5044
Processing	Logstash conditional routing on event_id
Transformation	Add logon_result. GeoIP on IpAddress.
Final Destination	Elasticsearch: logs-windows-YYYY.MM.DD
Visualization	Kibana — Logon success/failure, attack origins